

Blockchain-Based PKI — Complete Execution Guide

Zero cost. No prior blockchain experience assumed. Start to finish.

This guide takes you from an empty computer to a working, demo-ready, deployed project that matches your proposal document and your slide deck exactly.

Part 0 — What you are actually building (plain English)

Forget blockchain for a second. Here is the system in one paragraph:

A user opens a web page. The page generates a cryptographic key pair inside their browser — a **private key** (secret, saved to their own computer) and a **public key** (safe to share). The user clicks "Issue Certificate." The public key plus their name and an expiry date get written into a smart contract on Ethereum. Anyone in the world can now type that certificate's ID into a "Verify" page and the blockchain tells them: VALID, EXPIRED, or REVOKED. If the user's private key is ever stolen, they click "Revoke" and every future verification instantly returns REVOKED.

There is no Certificate Authority anywhere in that story. That is the whole point of your project, and that is your answer in the viva.

Five components (matches Section 4.1 of your document):

Layer	What it is in practice
User Layer	A React dashboard (<code>dashboard.html</code>)
Processing Layer	JavaScript using <code>ethers.js</code> + the Web Crypto API
Logic Layer	<code>BlockchainPKI.sol</code> — your Solidity smart contract
Consensus Layer	Ethereum Sepolia test network
Storage	Contract state variables (permanent, public)

Part 1 — The completely free toolchain

Everything below is free forever. Nothing needs a credit card.

Tool	Purpose	Cost	Install needed?
Visual Studio Code	Your code editor — where you write everything	Free	Yes
Node.js (LTS)	Runs JavaScript on your computer; Hardhat needs it	Free	Yes
Hardhat	Compiles, tests and deploys your Solidity contract from the terminal	Free	Yes (via npm)
Solidity extension for VS Code	Syntax highlighting + error squiggles while you type	Free	Yes (extension)
MetaMask (browser extension)	Your Ethereum wallet + the bridge between your web page and the blockchain	Free	Yes (extension)
Hardhat local network	A private blockchain on your own machine for development	Free	No (comes with Hardhat)
Sepolia testnet	A real, public Ethereum network that uses worthless test coins	Free	No
Sepolia faucet	Gives you free test ETH to pay gas	Free	No
ethers.js (via CDN + npm)	JavaScript library to talk to the contract	Free	No
React 18 (via CDN)	The dashboard's component framework	Free	No
Tailwind CSS (play CDN)	Styling the dashboard	Free	No
Babel standalone (via CDN)	Compiles JSX in the browser, so no build step	Free	No
GitHub + GitHub Pages	Hosting your web page on a real public URL	Free	No
draw.io (app.diagrams.net)	Architecture + workflow diagrams for the report	Free	No
Sepolia Etherscan	Public proof your contract exists — great for the demo	Free	No

Critical point about cost: you will deploy to **Sepolia**, not Ethereum mainnet. Sepolia ETH has *no monetary value* and is given away free. Your proposal's "high transaction fees" research gap

is addressed by noting that the design is gas-optimised and deployed on a test network; you never spend real money. Say exactly that in your report.

Why Hardhat instead of a browser IDE: Hardhat is the professional standard. It gives you three things a browser IDE cannot — **automated tests** written in JavaScript that run in two seconds and re-run every time you change the contract, a **reproducible deployment script** that your examiner can run themselves, and **everything in one VS Code project** that lives in Git alongside your frontend. That last point matters for your grade: a repository containing `contracts/`, `test/`, `scripts/` and `frontend/` reads as an engineering project rather than a class exercise.

Part 2 — Setup (Day 1, about 45 minutes)

Step 2.1 — Install MetaMask

1. Go to `metamask.io/download` and install the extension for Chrome, Edge, Brave or Firefox.
2. Click **Create a new wallet**. Set a password.
3. It shows you a 12-word **Secret Recovery Phrase**. Write it on paper.
 - This is a *test* wallet. Never put real money in it. Never paste the phrase into any website, ever. Never put it in your project report or GitHub.
4. Finish setup. You now have an Ethereum address that looks like `0x7a3f...9c21`. Copy it somewhere — this is your identity for the whole project.

Step 2.2 — Switch MetaMask to the Sepolia test network

1. Open MetaMask, click the network dropdown at the top left (it says "Ethereum Mainnet").
2. Turn ON "**Show test networks**" in Settings → Advanced if you don't see them.
3. Select **Sepolia**.
4. Your balance will read `0 SepoliaETH`. That's expected.

Step 2.3 — Get free test ETH

You need a tiny amount of test ETH to pay for transactions. Try these in order — faucet availability changes, so if one is down, use the next:

- Google Cloud Web3 Sepolia faucet
- Alchemy Sepolia faucet (free account signup)
- Chainlink faucet
- Infura Sepolia faucet
- POW faucet (mines test ETH in your browser — slow but needs no account)

Paste your MetaMask address, complete the captcha, wait 1-3 minutes. You need roughly **0.05 SepoliaETH**, which is far more than enough for this entire project. 0.5 is luxurious.

If every faucet fails you: do not panic, and do not delay the project. Skip ahead to **Appendix A — The 100% offline fallback**. Your project can be completed, demoed and graded with zero faucet access. Read Appendix A now so you know it exists, then come back.

Step 2.4 — Install Visual Studio Code

1. Go to `code.visualstudio.com` and download the version for your operating system. Install it with all the default options.
2. Open VS Code. You'll see a dark window with a sidebar of icons on the left.
3. Click the **Extensions** icon in that sidebar (it looks like four squares).
4. Search for **"Solidity"** by *Juan Blanco* and click **Install**. This gives you colour-coded Solidity code and red underlines when you make a mistake.
5. Optional but nice: also install **"Prettier - Code formatter"**.

The one VS Code concept you need: the **integrated terminal**. It's a command line built into the editor. Open it with **Ctrl + `**` (backtick, the key above Tab) on Windows/Linux, or `**Cmd +` on Mac**. You can also use the menu: **Terminal → New Terminal**. Every command in this guide gets typed in there.

Step 2.5 — Install Node.js

Hardhat is a Node.js program, so Node has to exist on your machine first.

1. Go to `nodejs.org` and download the **LTS** version (the left-hand button, the one that says "Recommended For Most Users").
2. Install it with all default options. On Windows, leave every checkbox as it is.
3. **Close and reopen VS Code completely**. This matters — VS Code only notices newly installed programs after a restart.
4. Open the terminal (**Ctrl + `**) and type:

```
bash
node --version
npm --version
```

You should see two version numbers, something like `v22.11.0` and `10.9.0`. If you get "command not found" or "not recognised", Node isn't installed properly or you didn't restart — do step 3 again.

(`npm`) stands for Node Package Manager. It came with Node. It's how you install Hardhat.)

Step 2.6 — Create your project folder

1. Make a folder on your computer called `blockchain-pki`. Put it somewhere simple like your Desktop or Documents.
 - **Avoid folder names with spaces or special characters** — Node tooling handles them badly. `blockchain-pki` is safe; `My Project (final)` is not.
2. In VS Code: **File** → **Open Folder** → select `blockchain-pki` → Open.
3. If VS Code asks "Do you trust the authors of the files in this folder?", click **Yes, I trust the authors**.
4. Open the terminal (**Ctrl + `**`). The prompt should already be inside your `blockchain-pki` folder. Confirm with:

```
bash
```

```
pwd
```

(On Windows PowerShell, `pwd` works too.) It should print a path ending in `blockchain-pki`.

Step 2.7 — Install Hardhat

Type these one at a time in the VS Code terminal, pressing Enter after each and waiting for it to finish.

```
bash
```

```
npm init -y
```

This creates a `package.json` file — a manifest listing what your project uses. You'll see it appear in the VS Code sidebar.

```
bash
```

```
npm install --save-dev hardhat @nomicfoundation/hardhat-toolbox dotenv
```

This is the big one. It downloads Hardhat and its testing tools. **It will take 2–5 minutes** and print a lot of text. A folder called `node_modules` appears — that's normal, it holds the downloaded libraries, and you will never open it or upload it to GitHub.

You may see warnings in yellow about "deprecated" packages. **Ignore them**. Only red `ERR!` lines are real problems.

Now initialise Hardhat:

```
bash  
npx hardhat init
```

(`npx`) runs a program you just installed, without installing it globally.)

It asks a few questions. Answer:

- **What do you want to do?** → Use the arrow keys to select "Create a JavaScript project" → Enter
- **Hardhat project root?** → just press Enter (accepts your current folder)
- **Add a .gitignore?** → y → Enter
- **Install sample project dependencies?** → y → Enter

When it finishes you'll have this structure in the VS Code sidebar:

```
blockchain-pki/  
├─ contracts/      ← your Solidity goes here  
├─ scripts/        ← your deployment script goes here  
├─ test/           ← your automated tests go here  
├─ node_modules/   ← downloaded libraries (ignore this)  
├─ hardhat.config.js ← Hardhat's settings file  
├─ package.json  
└─ .gitignore
```

Delete the sample files Hardhat created so they don't confuse you later:

```
bash  
rm contracts/Lock.sol test/Lock.js
```

(On Windows PowerShell use `del contracts\Lock.sol` and `del test\Lock.js`, or just right-click each file in the VS Code sidebar → Delete.)

Finally, confirm Hardhat is alive:

```
bash  
npx hardhat --version
```

A version number means you're ready. Setup done.

Part 3 — Build the smart contract (Week 4 of your timeline)

Step 3.1 — Create the file

In the VS Code sidebar, right-click the `contracts` folder → **New File** → type `BlockchainPKI.sol` → Enter.

Step 3.2 — Paste the contract

Paste the entire contents of the `BlockchainPKI.sol` file provided alongside this guide, then save with **Ctrl + S**. Read the comments in it — they explain each function line by line, and those comments are the raw material for your report's "Implementation" section.

What the contract does, function by function:

Function	Purpose	Your objective it satisfies
<code>issueCertificate(name, publicKey, validityDays)</code>	Stores a new certificate on-chain, owned by whoever sent the transaction	"Securely store users' public keys on the blockchain" / "Certificate generation"
<code>verifyCertificate(id)</code>	Returns VALID / EXPIRED / REVOKED / NOT_FOUND. Free — costs no gas	"Verify certificates without depending on a central authority"
<code>verifyByFingerprint(hash)</code>	Look up a certificate by the hash of a public key	Verification
<code>revokeCertificate(id, reason)</code>	Owner-only. Flips the certificate to revoked forever	"Allow certificates to be revoked when needed"
<code>getCertificatesByOwner(addr)</code>	Lists all certificate IDs owned by an address	User dashboard
<code>getCertificate(id)</code>	Full certificate record	Transparency
<code>totalCertificates()</code>	Count, for statistics on your demo	Nice-to-have

The decentralisation argument (memorise this for your viva): there is no `owner`, no `admin`, and no `onlyAdmin` modifier anywhere in this contract. Any Ethereum address can issue a certificate for itself, and only that same address can revoke it — enforced by `require(c.owner`

`== msg.sender)`). There is deliberately no super-user who can revoke someone else's certificate. That is what makes it a *decentralized* PKI rather than a CA with extra steps.

Step 3.3 — Configure the compiler

Open `hardhat.config.js` in VS Code. Replace everything in it with the contents of the `hardhat.config.js` file provided alongside this guide, then save.

The important line is the Solidity version. Your contract begins with `pragma solidity ^0.8.20;`, so the config must say `version: "0.8.20"` (or higher). If these two disagree, compilation fails with a message about an unresolved pragma.

Step 3.4 — Compile

In the VS Code terminal:

```
bash  
  
npx hardhat compile
```

You want to see:

```
Compiled 1 Solidity file successfully
```

A new `artifacts/` folder appears. Inside it, Hardhat has written `artifacts/contracts/BlockchainPKI.sol/BlockchainPKI.json` — that file contains the **ABI** and the compiled bytecode. You'll come back for the ABI later.

Reading Solidity errors: Hardhat prints the file, the line number, and the problem. In practice 95% of them are a missing semicolon or an unclosed brace on the line named, or the line just above it. Fix it and run `npx hardhat compile` again. It only recompiles what changed, so it's fast.

If you want Hardhat to recompile automatically every time you save, run `npx hardhat watch` — but plain `compile` is fine and one less thing to go wrong.

Step 3.5 — Write automated tests (this is the big upgrade)

This is what Hardhat gives you that a browser IDE cannot: **your twelve test cases become code that runs in two seconds, every time, forever.** You never click through them by hand again, and you cannot forget one.

Right-click the `test` folder → **New File** → `BlockchainPKI.test.js`. Paste the contents of the `BlockchainPKI.test.js` file provided alongside this guide. Save.

You don't need to fully understand the test syntax to use it, but here is the shape of it so the code isn't a black box:

- `describe(..., ...)` — groups related tests together
- `it("should do X", ...)` — one individual test case
- `beforeEach(...)` — runs before every single test, deploying a clean contract so no test can contaminate another
- `expect(result).to.equal(5)` — the assertion; the test passes if this is true
- `await expect(tx).to.be.revertedWith("message")` — asserts a transaction **fails** with a specific error. This is how you test that your security rules actually work
- `ethers.getSigners()` — Hardhat hands you 20 pre-funded fake accounts. The tests use three of them as Alice, Bob and Carol

Step 3.6 — Run the tests

```
bash  
  
npx hardhat test
```

Hardhat silently spins up a fresh local blockchain in memory, deploys your contract, runs everything, and tears it down. Expected output:

```
BlockchainPKI
  Deployment
    ✓ starts with zero certificates
  Issuance
    ✓ issues a certificate and assigns ID 1
    ✓ records the caller as the owner
    ✓ rejects a duplicate public key
    ✓ rejects an empty subject name
    ✓ rejects zero-day validity
    ✓ rejects validity over 3650 days
    ✓ emits CertificateIssued
  Verification
    ✓ reports a fresh certificate as VALID
    ✓ reports an unknown ID as NOT_FOUND
    ✓ reports an expired certificate as EXPIRED
    ✓ finds a certificate by fingerprint
  Revocation
    ✓ lets the owner revoke
    ✓ blocks a non-owner from revoking
    ✓ blocks double revocation
    ✓ reports a revoked certificate as REVOKED
    ✓ emits CertificateRevoked
  Ownership
    ✓ lists all certificates owned by an address
```

18 passing (1s)

Screenshot that output. Eighteen green ticks is a far stronger Testing chapter than twelve manual screenshots, and it took two seconds instead of twenty minutes. Put the screenshot in your report next to a table mapping each test name to the requirement it verifies.

Two of those tests deserve a mention in your viva because they prove your security claims rather than your happy path:

- **"blocks a non-owner from revoking"** — proves the decentralisation argument. Bob cannot touch Alice's certificate. The contract rejects the transaction.
- **"reports an expired certificate as EXPIRED"** — this one uses Hardhat's time-travel feature, `time.increase()`, to jump the blockchain's clock forward by a year. You physically cannot test expiry any other way without waiting a year, and that capability alone justifies using Hardhat.

If a test fails, the output tells you the expected value, the actual value, and the line. Fix and re-run.

Step 3.7 — Optional: check your gas costs

```
bash  
  
REPORT_GAS=true npx hardhat test
```

(On Windows PowerShell: `$env:REPORT_GAS="true"; npx hardhat test`)

This prints a table of how much gas each function consumes. Drop it straight into your report's "Economic Feasibility" section — it is concrete evidence for your claim that verification is free and only issuance and revocation cost anything.

Step 3.8 — Optional: deploy to a local node and click around

If you want to interact with the contract by hand the way a browser IDE lets you, Hardhat has a console. Run:

```
bash  
  
npx hardhat console
```

Then type JavaScript at the `>` prompt:

```
javascript  
  
const F = await ethers.getContractFactory("BlockchainPKI");  
const c = await F.deploy();  
await c.issueCertificate("Alice", "TESTKEY001", 365);  
await c.verifyCertificate(1);  
await c.totalCertificates();
```

Press **Ctrl + C** twice to exit. This is handy for poking at something quickly, but the test file is where your real verification lives.

Part 4 — Deploy to the real Sepolia network (Week 6)

Now you put it on a genuinely public blockchain that your examiner can look up.

Step 4.1 — Export your MetaMask private key

Hardhat deploys from the terminal, so it needs a key to sign the deployment transaction with. You have to give it your test wallet's private key.

Read this before you do it. A private key gives total control of a wallet. This is safe *only* because this wallet is a throwaway test wallet holding worthless Sepolia ETH. **Never** do this for a wallet that holds real money, and **never** commit the key to GitHub. The steps below make sure it can't be.

1. Open MetaMask → click the three dots next to your account name → **Account details**.
2. Click **Show private key**. Enter your MetaMask password.
3. Copy the long hex string.

Step 4.2 — Store it safely in a `.env` file

1. In the VS Code sidebar, right-click the **root** of `blockchain-pki` (not inside a folder) → **New File** → name it exactly `.env` (with the leading dot, and no extension).
2. Put this in it, pasting your own values:

```
PRIVATE_KEY=your_private_key_here_without_the_0x_prefix  
SEPOLIA_RPC_URL=https://ethereum-sepolia-rpc.publicnode.com
```

3. **Now the critical step.** Open the `.gitignore` file in your project root and make sure it contains these three lines:

```
node_modules  
.env  
artifacts
```

`.gitignore` tells Git which files to never upload. Without `.env` in that list, your private key would go public the moment you push to GitHub. **Check this now, not later.**

4. Confirm it worked: in the VS Code sidebar, `.env` should appear greyed out or faded compared to your other files. That's VS Code showing you it's ignored.

The `hardhat.config.js` provided reads these values via the `dotenv` package you already installed, so nothing else needs changing.

Step 4.3 — Add the deployment script

Right-click the `scripts` folder → **New File** → `deploy.js`. Paste the contents of the `deploy.js` file provided alongside this guide. Save.

The script does four things: deploys the contract, waits for confirmation, prints the address, and **writes the address and ABI into a file your frontend can read** — so you don't have to copy-paste them by hand.

Step 4.4 — Do a dry run on your local network first

Never deploy to a public network before testing the script locally. Hardhat can run a real blockchain on your own machine.

Open a **second terminal** in VS Code (click the + icon in the terminal panel) and run:

```
bash  
npx hardhat node
```

This starts a local Ethereum node at `http://127.0.0.1:8545` and prints 20 test accounts with 10000 fake ETH each. **Leave this terminal running** — it's a server.

Switch back to your first terminal and run:

```
bash  
npx hardhat run scripts/deploy.js --network localhost
```

You should see the contract deploy and print an address in about a second. If this works, your script is correct, and you haven't spent a single test ETH finding that out.

Stop the node with **Ctrl + C** when you're done.

Step 4.5 — Deploy to Sepolia for real

Make sure your wallet has Sepolia ETH (Step 2.3), then:

```
bash  
npx hardhat run scripts/deploy.js --network sepolia
```

Wait 15-45 seconds. You'll see something like:

```
Deploying BlockchainPKI...
Deployer: 0x7a3f...9c21
Balance: 0.4982 ETH

✓ BlockchainPKI deployed to: 0x9fE4b2A7c1D38e5F06a4B9c2D7e1F8a3B2c5D6E9

Network:      sepolia (chainId 11155111)
Transaction: 0x4c1b...
Etherscan:    https://sepolia.etherscan.io/address/0x9fE4...

✓ Wrote frontend/contract-address.json
```

Save that contract address. Everything downstream needs it. The script also wrote it to `frontend/contract-address.json` for you.

Verify it publicly on Etherscan (do this — it impresses reviewers)

1. Go to `sepolia.etherscan.io`.
2. Paste your contract address into the search bar.
3. You'll see your contract, its creation transaction, and every future transaction — permanently, publicly, timestamped.

Screenshot this page. It is the single most convincing slide in your presentation: *"here is our system, live on a public blockchain, verifiable by anyone, right now."*

Publishing your source code on Etherscan takes it further — examiners can read your Solidity directly on a third-party site, and Etherscan generates a clickable interface for your contract. Hardhat can do this in one command.

Get a free API key from `etherscan.io/myapikey` (free account signup), add it to your `.env`:

```
ETHERSCAN_API_KEY=your_key_here
```

Then run:

```
bash

npx hardhat verify --network sepolia YOUR_CONTRACT_ADDRESS
```

After a minute, refresh your Etherscan page. Your source code is now displayed publicly with a green checkmark, plus **Read Contract** and **Write Contract** tabs anyone can use. This is a genuinely impressive five minutes of work.

(If the API key step is fiddly, skip it. It's optional polish, not a requirement.)

Part 5 — The React dashboard (Week 5)

You have been given `dashboard.html` — a complete React application: sidebar navigation, six views, live blockchain reads, a demo mode, and a designed certificate component. This is your primary frontend.

It is **React 18 with hooks, components and context**, styled with **Tailwind CSS** — but delivered as a single file that runs by double-clicking it. No Node.js, no npm install, no `node_modules`, no build step, no deployment pipeline. React, Tailwind and ethers.js all load from free CDNs, and Babel compiles the JSX in the browser.

(You also have the simpler `index.html` from before. Keep it as a backup — if anything ever goes wrong with the React version five minutes before your demo, it does the same job with zero dependencies.)

Step 5.1 — Put it in your project and insert the contract address

1. In the VS Code sidebar, right-click the project root → **New Folder** → `frontend`.
2. Drag `dashboard.html` into it (or right-click → New File and paste the contents).
3. Open it, and near the top of the last `<script>` block find:

```
javascript
```

```
const CONTRACT_ADDRESS = "PASTE_YOUR_CONTRACT_ADDRESS_HERE";
```

Replace it with the address your deploy script printed in Step 4.5. Save with **Ctrl + S**. **That is the only edit required.**

Your project now looks like this:

```
blockchain-pki/
├── contracts/BlockchainPKI.sol
├── test/BlockchainPKI.test.js
├── scripts/deploy.js
├── frontend/
│   ├── dashboard.html      ← your React app
│   └── contract-address.json ← written by deploy.js
├── hardhat.config.js
├── .env                    ← never committed
└── .gitignore
```

That is a clean, professional repository layout, and it's worth a screenshot in your report's Implementation chapter.

Why isn't the address imported automatically? It could be — `deploy.js` writes `contract-address.json` right next to the dashboard. But reading a local JSON file from a `file://` page is blocked by browser security, so hard-coding the one constant is simpler and cannot fail. If you later serve the frontend over `http://`, you can `fetch('./contract-address.json')` instead.

Step 5.2 — Run it

Right-click `dashboard.html` in the VS Code sidebar → **Reveal in File Explorer** (or **Reveal in Finder**), then double-click it. It opens in your browser and works immediately.

Better option — VS Code's Live Server. Install the "Live Server" extension by Ritwick Dey, then right-click `dashboard.html` → **Open with Live Server**. This serves the page over `http://127.0.0.1:5500`, which MetaMask detects far more reliably than `file://`, and it auto-refreshes the browser every time you save. For frontend work this is worth the thirty seconds it takes to install.

If you'd rather not install anything: open a terminal in the `frontend` folder and run `npx http-server -p 8000`, then visit `http://localhost:8000`. Or skip straight to Part 7 and use GitHub Pages.

Step 5.3 — Demo mode (read this before your presentation)

There is a **Demo mode** toggle at the bottom of the sidebar. When it's on, the app runs on sample certificates with no network calls at all — one valid, one expired, one revoked, so every visual state is on screen.

Three reasons this matters:

1. You can build, style and rehearse the UI before your contract is even deployed.
2. You can show all three certificate states without waiting for one to expire.
3. **If the venue Wi-Fi dies during your review, your demo still runs.** This has saved more project presentations than any other single feature. Turn it on, and the app never touches the network.

If you leave `CONTRACT_ADDRESS` unset, demo mode turns on automatically.

Step 5.4 — The six views

View	What it does	Lifecycle step
Overview	Dashboard home. Features one live certificate as a document, the six-step lifecycle, and valid/expired/revoked counts	—
Keys	Generates an ECDSA P-256 key pair via the browser's Web Crypto API; downloads the private key as JSON	1–2
Issue	Form plus a live preview of the certificate as you type. Writes to the contract	3–4
Certificates	All your certificates as document cards, filterable by status, with a revocation dialog	—
Verify	Public lookup by number. Large seal verdict. No wallet required	5
Signatures	Sign a message with your private key; verify any signature against the on-chain public key	5

Step 5.5 — The design, and how to talk about it in your review

Faculty do ask "why does it look like this?" Have an answer. Here is yours.

The palette comes from the subject matter, not from a dashboard template. Certificates are physical objects: paper, ruled borders, wax seals, patina on old official stamps. So the three status colours are **verdigris** (the green patina on aged copper seals) for valid, **brass** for expired, and **seal red** for revoked. The palette *is* the status system — colour carries information rather than decorating.

Certificates are drawn as certificates. Each one is a document card with a coloured status rule along the top, a guilloche pattern behind it (the fine engine-turned line work printed on real certificates and banknotes — done here in pure CSS, nothing downloaded), the subject name set in a serif the way a real certificate would be, and a circular **seal medallion** stamped in the corner.

One moment of motion, not scattered effects. The seal stamps down with a slight overshoot and rotation when a certificate appears — like a stamp hitting paper. Nothing else animates. It respects `(prefers-reduced-motion)`.

Typography does two jobs. Source Serif carries certificate content and verdicts, because that is the register of a formal document. IBM Plex Sans carries the interface. IBM Plex Mono is

used *only* for genuine cryptographic material — addresses, fingerprints, base64 keys — where character alignment actually helps you read a hash, not as decoration on labels.

Restraint everywhere else. The shell is a quiet deep navy; the workspace is a neutral cool grey; cards are plain. All the visual weight is spent on the one thing that matters — the certificate and its status.

Step 5.6 — Optional: convert to a real Vite build (only if asked)

If your faculty specifically requires a standard React project structure with `npm`, the migration is mechanical and still completely free:

```
bash

npm create vite@latest pki-dashboard -- --template react
cd pki-dashboard
npm install
npm install ethers
npm install -D tailwindcss @tailwindcss/vite
npm run dev
```

Then move the component functions out of `dashboard.html` into `src/App.jsx`, move the `tailwind.config` theme block into your Tailwind config, and delete the CDN `<script>` tags. Build with `npm run build` and deploy the `dist/` folder to GitHub Pages.

Do this only if it is required. It adds a Node.js install, a build step and a class of errors you do not currently have, in exchange for no new functionality in a project this size. The single-file version is real React — same hooks, same components, same context API — and it cannot break on a machine that isn't yours.

Part 6 — The killer demo feature: prove key ownership (optional but do it)

This is what separates a B project from an A project, and it takes about ten minutes because the code is already written for you.

The **Signatures** view in `dashboard.html` demonstrates the actual *purpose* of PKI, not just certificate storage:

1. Alice types a message: `"Transfer approved – Alice"`.
2. She loads her downloaded private key file and signs the message. A signature string appears.
3. Bob (the verifier) enters Alice's **certificate ID**, the **message**, and the **signature**.

4. The app fetches Alice's public key **from the blockchain**, checks the certificate isn't revoked or expired, and cryptographically verifies the signature.
5. Result: **AUTHENTIC** ✓ or **INVALID** ✗.

Then the money shot for your demo: **Alice revokes her certificate, Bob tries the exact same valid signature again, and it now returns REJECTED — CERTIFICATE REVOKED.**

That single sequence demonstrates registration, storage, issuance, verification, and revocation end-to-end, with real cryptography, in under ninety seconds. Build your entire live demo around it.

Part 7 — Free public hosting with GitHub Pages (Week 6)

Your project should have a real URL, not a file on your laptop. And since you now have a proper project folder, you'll push the whole thing to Git rather than dragging files into a web form.

Step 7.1 — One safety check first

Open `.gitignore` in VS Code and confirm it contains, at minimum:

```
node_modules
.env
artifacts
cache
```

Do not skip this. `.env` holds your private key. Once something is pushed to a public GitHub repo, treat it as permanently public even if you delete it afterwards.

Step 7.2 — Create the repository

1. Create a free account at `github.com`.
2. Click + → **New repository**. Name it `blockchain-pki`. Set it to **Public**. **Do not** tick "Add a README" — your local folder will provide the files. Click **Create repository**.
3. GitHub shows you a page of commands. Ignore it; use the ones below.

Step 7.3 — Push from VS Code's terminal

Install Git first if you don't have it (`git-scm.com`, default options, then restart VS Code). Check with `git --version`.

In your project terminal:

```
bash
```

```
git init
git add .
git commit -m "Blockchain PKI: contract, tests, deploy script, dashboard"
git branch -M main
git remote add origin https://github.com/YOURUSERNAME/blockchain-pki.git
git push -u origin main
```

Replace `YOURUSERNAME`. GitHub will ask you to sign in in a browser window.

Immediately after pushing, open your repo on github.com and confirm there is no `.env` file and no `node_modules` folder listed. If `.env` is there, delete the repository entirely, export a brand-new MetaMask account, and start over with a correct `.gitignore`. Better to lose ten minutes than to leak a key.

*(Prefer buttons to commands? VS Code's **Source Control** panel — the third icon in the sidebar — does all of this with a **Publish to GitHub** button. Use whichever you're comfortable with.)*

Step 7.4 — Turn on GitHub Pages

GitHub Pages serves a file called `index.html` from whichever folder you point it at. Your dashboard is `frontend/dashboard.html`, so rename it:

```
bash

git mv frontend/dashboard.html frontend/index.html
git commit -m "Rename dashboard for GitHub Pages"
git push
```

Then:

1. On your repo page, go to **Settings → Pages**.
2. Under "Source" choose **Deploy from a branch**.
3. Branch: **main**. Folder: **/frontend**. Click **Save**.
4. Wait 1-2 minutes and refresh. GitHub gives you a live URL:

`https://yourusername.github.io/blockchain-pki/`

That URL is now your project. It works on any device, anywhere, forever, for free. Put it on your title slide and in your report.

If `/frontend` isn't offered as a folder option, only `/` and `/docs` are available on your account. In that case rename the folder to `docs` instead (`git mv frontend docs`) and select `/docs`.

Step 7.5 — Why the whole repo matters for your grade

A repository containing `contracts/`, `test/`, `scripts/` and `frontend/` — with a real commit history — is itself evidence of engineering process. Add a `README.md` at the root with: what the project does, the live URL, the Sepolia contract address, and the three commands to run it (`npm install`), `npx hardhat test`, `npx hardhat run scripts/deploy.js --network sepolia`). That means your examiner can clone your repo and reproduce your entire project in under five minutes, which is the strongest possible answer to "does this actually work?"

Part 8 — Testing (Week 7)

Your contract layer is already covered by the 18 automated Hardhat tests from Part 3 — run `npx hardhat test`, screenshot the green output, and that's your unit testing section done. What follows is the end-to-end layer those tests can't reach, because it involves a browser, a wallet and a human:

#	Test case	Steps	Expected
E1	Wallet connection	Click Connect Wallet	Address + "Sepolia" displayed
E2	Wrong network guard	Switch MetaMask to Mainnet, reload	Warning: "Please switch to Sepolia"
E3	Key generation	Click Generate Key Pair	Public key shown, private key <code>.json</code> downloads
E4	Certificate issuance	Fill form, submit, confirm in MetaMask	Tx hash shown, new cert ID returned
E5	On-chain persistence	Reload the page entirely, open My Certificates	Certificate still there — proves blockchain storage, not browser storage
E6	Third-party verification	Open the GitHub Pages URL on your phone , verify the ID	VALID — with no wallet connected
E7	Signature verification	Sign a message, verify it	AUTHENTIC
E8	Tampered message	Change one character of the message, verify	INVALID
E9	Revocation	Revoke the certificate	Tx confirms, status → REVOKED
E10	Post-revocation check	Re-verify the same signature	REJECTED — CERTIFICATE REVOKED
E11	Unauthorised revocation	Switch MetaMask account, try to revoke someone else's cert	Transaction rejected by contract
E12	Non-existent certificate	Verify ID 99999	NOT_FOUND, handled gracefully
E13	Demo mode	Toggle Demo mode in the sidebar, open Certificates	Three sample certificates showing valid, expired and revoked states
E14	Responsive layout	Open the dashboard on a phone	Sidebar collapses behind a Menu button, cards stack, nothing overflows
E15	Status filtering	On Certificates, click each filter	Only matching certificates shown; empty state when none

#	Test case	Steps	Expected
E16	Live preview	Type in the Issue form	Preview certificate updates as you type

E5 and E6 are the two that actually *prove* decentralisation to a sceptical examiner. Make sure you can perform both live.

Part 9 — Diagrams for the report (Week 3)

Use **app.diagrams.net** (free, no account needed, exports PNG).

Diagram 1 — System Architecture (five layers, vertical stack):

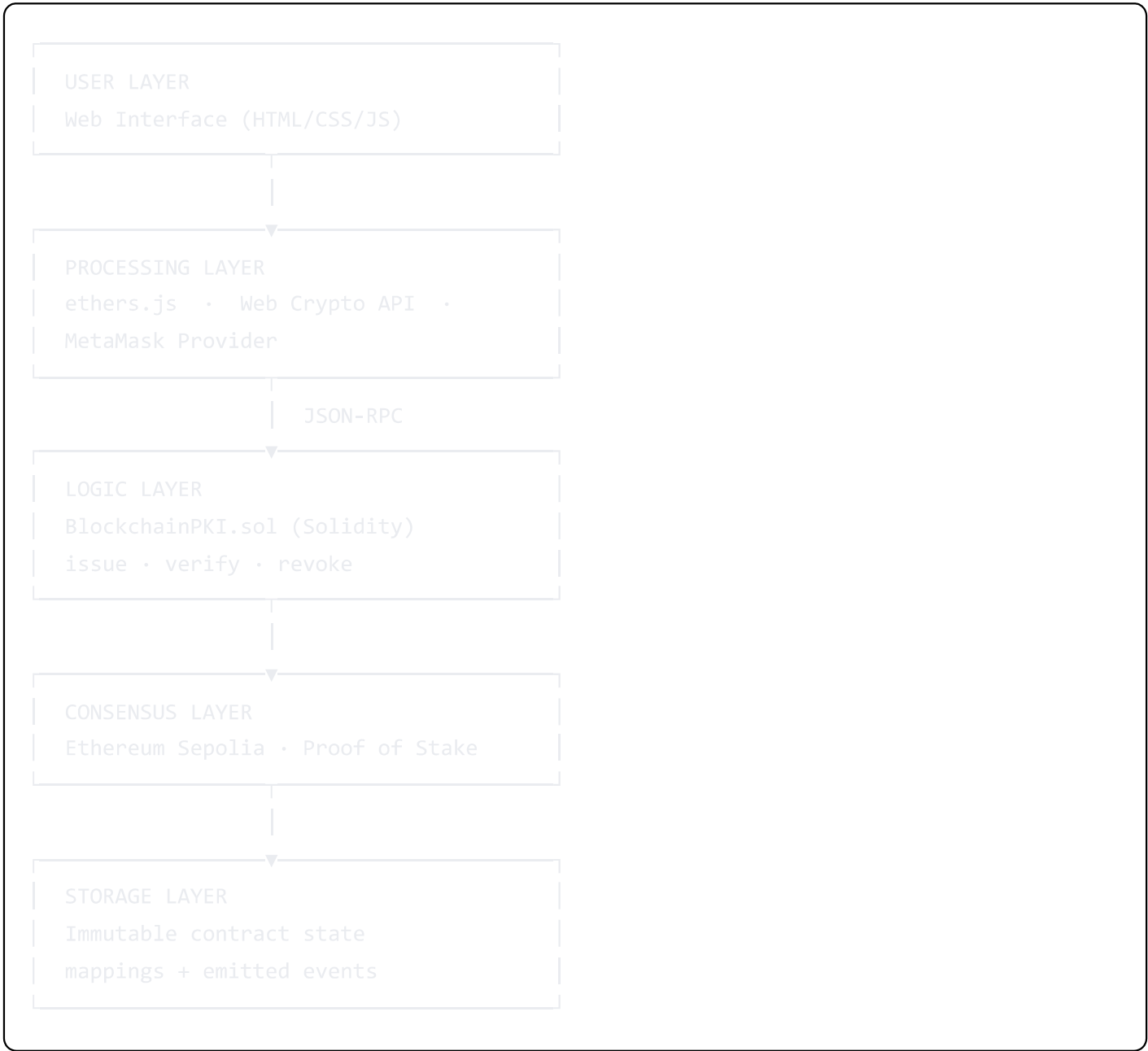
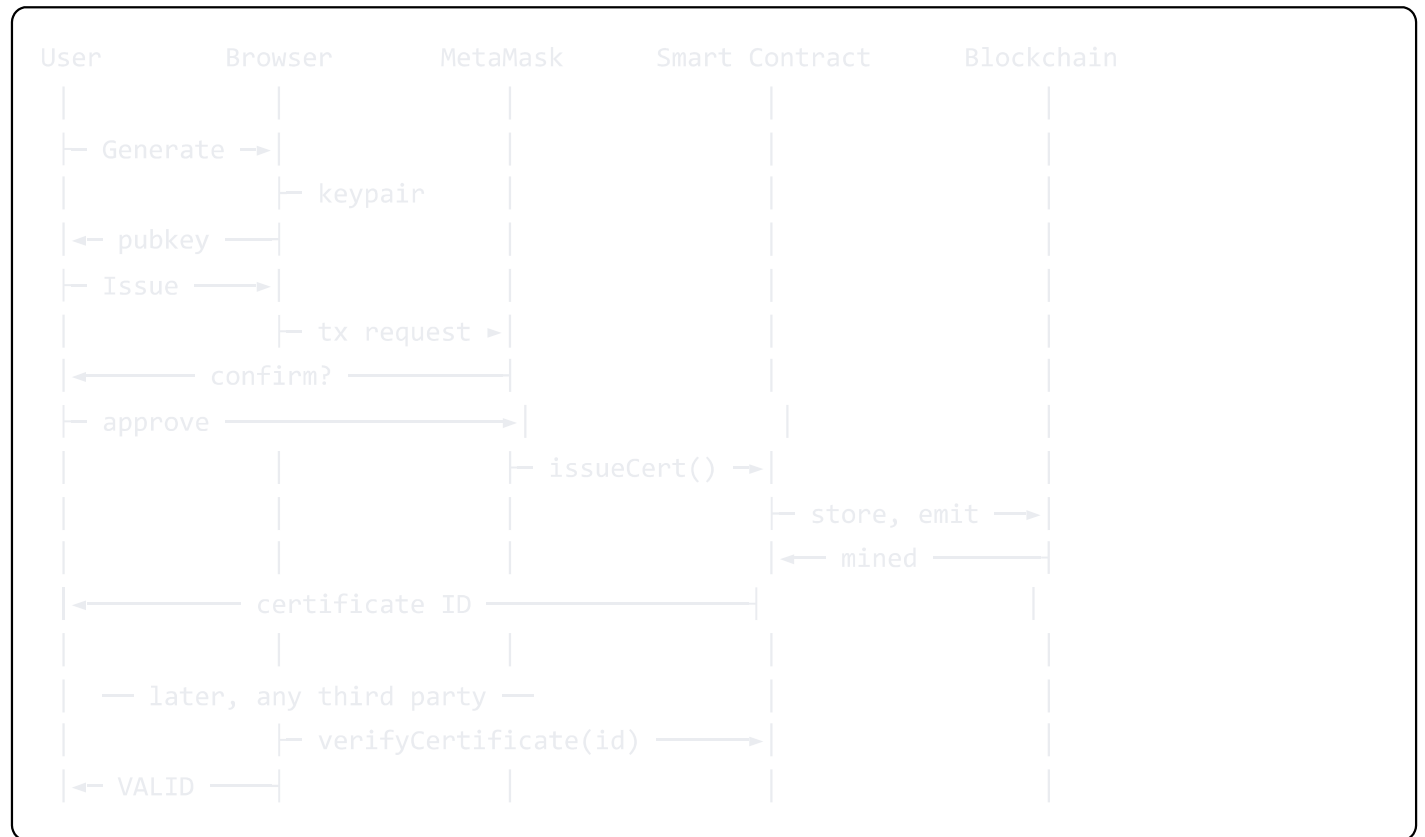


Diagram 2 — Certificate Lifecycle (circular, six steps): Registration → Key Generation → Blockchain Storage → Issuance → Public Verification → Revocation. This already exists on Slide 5 of your deck; redraw it cleanly in draw.io at higher resolution for the document.

Diagram 3 — Sequence Diagram (add this; faculty love sequence diagrams and your proposal doesn't have one yet):



Part 10 — Mapped onto your 8-week timeline

Your proposal already commits to 8 weeks. Here's what actually happens in each, with a concrete deliverable so you can prove progress at any checkpoint.

Week	Activity (from your proposal)	What you actually do	Deliverable
1	Literature survey	Read 6–8 papers on blockchain PKI, CA compromise (DigiNotar, Symantec), Certificate Transparency. Build a comparison table	Literature review + comparison table (Slide 3 already drafts this)
2	Requirement analysis	Write functional + non-functional requirements. Do the full Part 2 setup — MetaMask, VS Code, Node.js, Hardhat, faucet	Requirements doc, working Hardhat project, funded test wallet
3	System design	Draw all three diagrams from Part 9 . Design the contract's data structures on paper. Sketch the dashboard's component tree	Architecture, lifecycle, sequence diagrams + UI wireframe
4	Smart contract development	Part 3 . Write the contract, write the test file, get all 18 tests passing	<code>BlockchainPKI.sol</code> + <code>BlockchainPKI.test.js</code> + green test output
5	Frontend development	Part 5 . Run the React dashboard in Demo mode, walk every view, adjust copy and colours to taste	Working <code>dashboard.html</code> , screenshots of all six views
6	Blockchain integration	Part 4 + Part 7 . Local deploy dry run, then Sepolia, then push to GitHub and enable Pages	Live contract address + live URL + public repo
7	Testing	Part 8 . Run all 24 test cases, screenshot everything. Add Part 6 signature demo	Completed test report
8	Documentation & review	Write the final report, rehearse the demo, prepare viva answers	Report, slides, demo script

Realistic note: Weeks 4 and 5 are the ones that slip. Since you already have working code for both, you can compress them to 3–4 days each and bank the spare time for Week 7 testing, which is where marks are usually lost.

Part 11 — Viva preparation

The questions you *will* be asked, with the answers:

Q: Isn't this just a database with extra steps? No. A database has an administrator who can silently alter or delete a record. Our contract has no admin function — the code is public, deployed at a fixed address, and its history is append-only and independently auditable by anyone on Etherscan. Nobody, including us, can retroactively change a certificate's issuance time or un-revoke a revoked one.

Q: Where is the private key stored? Never on the blockchain, and never on any server. It is generated in the user's browser via the Web Crypto API and saved only to the user's own device. Only the public key is published. If we stored private keys on-chain, the system would be completely broken, because everything on a public blockchain is readable by everyone.

Q: Why Ethereum and not Hyperledger Fabric? Fabric is permissioned — you need an identity issued by a membership service to join. That reintroduces the central authority we are trying to eliminate. Our threat model is exactly "the central authority is compromised," so a public, permissionless chain is the correct fit. Fabric would be the right answer for an intra-organisational PKI; ours is public.

Q: What about gas costs on mainnet? Acknowledged in our research gap. Three mitigations: (1) verification is a `view` function and costs zero gas — only issuance and revocation cost anything, and those are rare, once-per-certificate-lifetime events; (2) we store a `bytes32` keccak fingerprint for lookups rather than searching strings; (3) production deployment would target an L2 such as Arbitrum or Polygon, where the same bytecode runs for a fraction of a cent.

Q: What happens if a user loses their private key? They can revoke the certificate from their wallet address (the wallet is separate from the certificate key pair) and issue a new one. This is the same model as SSH keys. Account recovery is deliberately out of scope, as stated in Section 1.3.

Q: How does someone actually know a certificate belongs to the real Alice? This is the name-binding problem, and it's honest to name it as a limitation. Our system guarantees that whoever controls a given Ethereum address issued a given public key, and that it hasn't been revoked. Binding that address to a real-world legal identity would require either an off-chain attestation or a web-of-trust endorsement layer — which we identify as future work.

Q: Why does the interface look the way it does? The design is derived from the subject matter rather than a generic dashboard template. Certificates are formal printed documents, so each one renders as a document — ruled border, guilloche line work, serif subject name, and a stamped seal. The three status colours are taken from that same world: verdigris patina for valid, brass for expired, and seal red for revoked, so colour carries the status information

instead of decorating the page. Monospace is reserved strictly for cryptographic material like hashes and keys, where character alignment aids reading.

Q: Why React, and why no build step? React gives us component reuse — the certificate document component is used in the overview, the list, the verification result and the live issue preview, so the four stay consistent by construction. We deliberately skipped a bundler because the app is a single page with no routing or code splitting to gain from one, and a build step would add an install dependency and a class of failure without adding functionality. It is standard React with hooks and context; the JSX is compiled in the browser.

Q: How do you know the contract is correct? We have 18 automated tests written in JavaScript with Hardhat and Chai, covering issuance, duplicate-key rejection, input validation, event emission, verification in all four states, and revocation access control. They run in about a second and re-run on every change, so a regression is caught immediately rather than discovered during a demo. Two tests specifically prove the security properties: one asserts a non-owner's revocation transaction reverts, and one uses Hardhat's block-time manipulation to fast-forward a year and confirm expiry is reported correctly — which we could not test any other way.

Q: Why Hardhat rather than a browser IDE? Three reasons. Tests become executable code instead of manual clicking, so they're repeatable and can't be forgotten. Deployment becomes a scripted, reproducible command our examiner can run themselves. And the contract, tests, deployment script and frontend live in one version-controlled repository, which is how real smart-contract projects are actually built.

Q: What's novel here? The combination in one lightweight interface: self-sovereign issuance with no admin role, on-chain revocation with instant global effect, and live signature verification against on-chain keys — with a UI simple enough for a non-technical user, which is the specific gap we identified in Section 2.2.

Future work section (write this in the report — it signals maturity)

- Web-of-trust endorsements: let addresses vouch for other certificates, with a trust score computed from endorsement depth
 - L2 deployment for real-world gas economics
 - ENS integration for human-readable identities
 - Merkle-tree batching to amortise gas across many certificates
 - Certificate Transparency-style public monitoring of issuance events
-

Part 12 — Troubleshooting

Symptom	Cause	Fix
"Insufficient funds for gas"	No Sepolia ETH	Use a faucet (Step 2.3), or fall back to Appendix A
MetaMask never pops up	Page opened as <code>file://</code>	Serve over <code>http://localhost</code> or use GitHub Pages
"Contract not deployed" / all reads return nothing	Wrong <code>CONTRACT_ADDRESS</code> , or wallet on the wrong network	Re-copy the address; confirm Sepolia is selected
Transaction stuck "pending" forever	Network congestion or too-low gas	In MetaMask: Activity → the tx → Speed up . Or Settings → Advanced → Clear activity tab data and retry
<code>npm</code> or <code>node</code> "not recognised"	Node installed but VS Code wasn't restarted	Close VS Code completely and reopen it
<code>npm run hardhat</code> says "not found"	You're in the wrong folder	<code>pwd</code> — you must be inside <code>blockchain-pki</code> , not its parent
Compile error about the pragma	<code>hardhat.config.js</code> version doesn't match the contract's <code>pragma</code>	Set <code>version: "0.8.20"</code> in the config
<code>Error HH8: invalid config</code>	A typo in <code>hardhat.config.js</code> , usually a missing comma or quote	Check the line number it names
<code>insufficient funds for intrinsic transaction cost</code> on deploy	Wallet funded on a different network, or empty	Confirm Sepolia balance in MetaMask
<code>invalid private key</code> / <code>bad hexlify value</code>	<code>.env</code> key still has the <code>0x</code> prefix, or a stray space/quote	Remove <code>0x</code> , remove quotes, no trailing spaces
<code>.env</code> values come out <code>undefined</code>	<code>.env</code> isn't in the project root, or the file is named <code>env</code>	Must be exactly <code>.env</code> , beside <code>hardhat.config.js</code>

Symptom	Cause	Fix
<code>npx hardhat run</code> hangs forever	Public RPC is rate-limiting you	Swap <code>SEPOLIA_RPC_URL</code> for a free Alchemy or Infura endpoint
<code>nonce too low</code> / <code>replacement underpriced</code>	A stuck earlier transaction	MetaMask → Settings → Advanced → Clear activity tab data , then retry
Tests fail after editing the contract	Stale build artifacts	<code>npx hardhat clean</code> then <code>npx hardhat compile</code>
Deploy worked but the dashboard reads nothing	<code>CONTRACT_ADDRESS</code> not updated after redeploying	Every redeploy creates a new address — update the constant
"Public key already registered"	You're reusing a test key	Generate a fresh key pair, or change one character
Faucet says "your mainnet account needs a balance"	Anti-bot rule on that specific faucet	Try a different faucet from the list, or the POW faucet
GitHub Pages shows 404	Not finished deploying, or file isn't named <code>index.html</code>	Wait 2 minutes; confirm the file is <code>frontend/index.html</code> and the Pages folder is <code>/frontend</code>
<code>ethers is not defined</code>	CDN blocked or offline	Check internet; or download <code>ethers.umd.min.js</code> and reference it locally
Dashboard shows a blank white page	A JSX syntax error	Press F12 → Console. Babel reports the exact line. Usually an unclosed tag or a missing <code>}</code>
Page takes 2–3 seconds to appear	Babel is compiling the JSX in the browser	Normal and expected. If it bothers you, do the Vite conversion in Step 5.6
Styling looks unstyled/plain	Tailwind CDN blocked (some campus networks filter it)	Switch on Demo mode and use a personal hotspot, or do the Vite conversion so Tailwind is bundled
Fonts look wrong	Google Fonts blocked	Harmless — the fallback stack takes over. Or self-host the two font files

Appendix A — The 100% offline fallback (if faucets fail)

Your project is not blocked by faucet availability. Use the **Hardhat network** — and this is a genuinely better fallback than the browser-IDE equivalent, because it lets you run the *full* system, frontend included.

The Hardhat network is a complete Ethereum node running on your own machine. It executes identical bytecode, enforces identical gas rules, and gives you 20 pre-funded accounts with 10,000 fake ETH each. The only difference from Sepolia is that it runs locally instead of being publicly shared.

Running the entire project locally

Terminal 1 — start the node (leave this running):

```
bash  
  
npx hardhat node
```

It prints 20 accounts with their addresses *and private keys*. Copy the private key of **Account #0**.

Terminal 2 — deploy to it:

```
bash  
  
npx hardhat run scripts/deploy.js --network localhost
```

Paste the printed address into `(CONTRACT_ADDRESS)` in your dashboard.

Connect MetaMask to your local node:

1. MetaMask → network dropdown → **Add network** → **Add a network manually**
2. Network name: `(Hardhat Local)`
3. New RPC URL: `(http://127.0.0.1:8545)`
4. Chain ID: `(31337)`
5. Currency symbol: `(ETH)` → Save
6. Import one of the node's test accounts: MetaMask → account menu → **Import account** → paste the private key of Account #0. You now have 10,000 fake ETH.

You also need to change one line in `(dashboard.html)`, since it's currently hard-coded to expect Sepolia:

```
javascript
```

```
const SEPOLIA_CHAIN = 31337n; // was 11155111n
const PUBLIC_RPC     = "http://127.0.0.1:8545";
```

Your entire dashboard now works end to end with no internet connection at all. Key generation, issuance, verification, signing, revocation — all of it. This is strictly more capable than the browser-IDE fallback, which could only exercise the contract, not the UI.

What to say in your report

"All functional testing was conducted against the Hardhat Network, a local Ethereum node that executes identical EVM bytecode to mainnet. This ensures reproducible results independent of public testnet faucet availability and network congestion. The system was additionally deployed to the Sepolia public testnet to demonstrate operation on a genuinely decentralized network."

That is true, professional, and defensible — and it's the standard practice for real smart-contract development, not a workaround.

Remember to change `SEPOLIA_CHAIN` **and** `PUBLIC_RPC` **back** before demoing on Sepolia.

Keeping two copies of the dashboard — `index.html` for Sepolia and `local.html` for offline — saves you from doing this under pressure.

Your master checklist

Setup

- ☐ MetaMask installed, wallet created, recovery phrase written on paper
- ☐ Switched to Sepolia network
- ☐ Test ETH received (or Appendix A plan confirmed)
- ☐ VS Code installed, Solidity extension added
- ☐ Node.js installed — `node --version` prints a number
- ☐ `blockchain-pki` folder created and opened in VS Code
- ☐ Hardhat installed — `npm install hardhat` prints a number
- ☐ Sample `Lock.sol` / `Lock.js` deleted

Contract

- ☐ `BlockchainPKI.sol` created in `contracts/`
- ☐ `hardhat.config.js` replaced, Solidity set to 0.8.20
- ☐ `npm run compile` succeeds

- ☐ `BlockchainPKI.test.js` created in `test/`
- ☐ `npx hardhat test` — all 18 pass, screenshot saved
- ☐ (Optional) Gas report captured with `REPORT_GAS=true`

Deployment

- ☐ `.env` created with `PRIVATE_KEY` (no `0x`) and `SEPOLIA_RPC_URL`
- ☐ `.gitignore` contains `.env`, `node_modules`, `artifacts`, `cache`
- ☐ `deploy.js` added to `scripts/`
- ☐ Local dry run works: `--network localhost`
- ☐ Deployed to Sepolia, contract address saved
- ☐ Found on Sepolia Etherscan, screenshot saved
- ☐ (Optional) Source verified with `npx hardhat verify`

Frontend

- ☐ `CONTRACT_ADDRESS` replaced in `dashboard.html`
- ☐ Demo mode explored — all six views walked through
- ☐ Wallet connects, header shows address and Sepolia
- ☐ Key generation works, private key downloads
- ☐ Certificate issues successfully, appears on Overview
- ☐ Certificates view lists and filters correctly
- ☐ Verify works from a second device with no wallet
- ☐ Sign & verify works, including the tampered-message case
- ☐ Revocation works and flips verification to Rejected
- ☐ Checked on a phone-sized screen
- ☐ Screenshots captured of every view for the report

Repository

- ☐ `.env` confirmed **absent** from the GitHub repo after pushing
- ☐ GitHub repo public, with `contracts/`, `test/`, `scripts/`, `frontend/`
- ☐ `README.md` written with live URL, contract address, and run commands
- ☐ GitHub Pages live, URL saved

Documentation

- ☐ Three diagrams drawn in draw.io
- ☐ 18 automated tests + 16 end-to-end cases documented with screenshots
- ☐ Report written with contract address + live URL + repo link included

- ☐ Slides updated with Etherscan screenshot and test output screenshot
 - ☐ Demo rehearsed twice, end to end, under 5 minutes
 - ☐ Viva answers from Part 11 reviewed
-

Recommended demo script (5 minutes)

1. **(30s)** "Traditional PKI trusts a single Certificate Authority. In 2011 DigiNotar was breached and issued fraudulent Google certificates. One compromise, millions of users affected." — Slide 2.
2. **(30s)** Open your live GitHub Pages URL. Connect MetaMask.
3. **(45s)** Generate a key pair. Point out: *"the private key just downloaded to my machine. It will never touch the blockchain."*
4. **(60s)** Issue a certificate. Confirm in MetaMask. Show the transaction hash.
5. **(45s)** Open **sepolia.etherscan.io**, paste the contract address, show the transaction you just made, live and public.
6. **(45s)** Hand your phone to a classmate. Have them verify the certificate ID on the same URL, with no wallet. VALID.
7. **(45s)** Sign a message. Verify it → AUTHENTIC. Change one character → INVALID.
8. **(30s)** Revoke the certificate. Re-verify the original valid signature → REJECTED, CERTIFICATE REVOKED.
9. **(30s)** "No certificate authority was involved at any point in that demonstration."

Insurance: before you walk into the room, open the dashboard once and confirm **Demo mode** works. If the Wi-Fi fails mid-presentation, toggle it on and continue the same walkthrough with sample data. Say plainly that you're switching to offline sample data — examiners respect a contingency plan far more than they penalise one.

Close there. That last line is the entire project in nine words.